

Projecte Final: Robot Inspector

Berta Vidal i Berta Aulet

Per dur a terme aquest projecte final de robòtica mòbil, hem decidit fer un programa per tal que el nostre robot sigui capaç d'actuar com un **investigador autònom** dins d'un **entorn desconegut**.

El comportament principal del robot es basa en la detecció i inspecció d'obstacles. Quan el robot es mou per l'entorn, utilitza els sensors per detectar qualsevol objecte que trobi davant seu. Si el que troba és una paret, simplement l'evita. Si, en canvi, el que detecta és un obstacle, el distingeix com a nou, si no l'ha detectat prèviament, o com a conegut si ja ha estat inspeccionat.

En el cas que l'obstacle sigui nou, el robot activa una rutina d'inspecció que consisteix a fer una volta completa sobre si mateix, durant la qual registra les coordenades globals d'aquell obstacle. Aquesta informació li permet identificar-lo en trobades futures i evitar inspeccions innecessàries. A més, en detectar un obstacle nou, el robot envia un missatge: "Nou obstacle detectat a x, y", indicant la seva localització. Un cop l'objecte ha estat inspeccionat i registrat, el robot continua el seu camí.

En canvi, si el robot torna a detectar un obstacle ja conegut, l'esquiva i envia el missatge: "Evitant obstacle conegut".

Gràcies a aquest funcionament, el robot pot gestionar de manera intel·ligent el seu entorn, reconeixent i actuant de manera diferent davant obstacles nous o coneguts, i garantint una navegació eficient.

Explicació del codi:

Per tal d'assolir el comportament descrit, hem completat la implementació que explicarem a continuació. Ens centrarem en els punts claus per comprendre la lògica del codi complet.

En primer lloc, hem decidit gestionar el funcionament del robot amb una estructura d'estats, un per cada una de les situacions en les quals es pot trobar. En concret, distingim cinc estats: 0 = moviment, 1 = detecció, 2 = paret detectada, 3 = obstacle conegut detectat, 4 = obstacle nou detectat. L'estat actual del robot s'emmagatzema en una variable global **estat**.

Pel que fa a l'estructura global del codi, distingim les funcions `odomCallback` i `scanCallback`, que gestionen directament la informació d'odometria i la rebuda pel sensor làser.

- **odomCallback**: actualitza la posició x, y i l'angle theta del robot.
- **scanCallback**: comprova un angle frontal de 20 graus per detectar qualsevol obstacle. En cas de trobar-lo, ho indica en una variable global **aprop**.

A més a més, inclou una regió de codi per la classificació de parets i obstacles. En concret, consulta un angle de 50 graus centrat en el punt on hi ha hagut una detecció i segueix una lògica d'ocupació per distingir obstacles individuals de parets (30 punts ocupats o més).

D'aquesta manera, aquestes dues funcions consulten la informació rebuda i la desen en variables globals. Per altra banda, la funció principal del codi és `on` es consulten aquestes variables i es defineix el comportament real del robot.

Així doncs, passem a l'explicació de la **funció main**, que està estructurada segons l'estat actual en què es troba el robot.

Estat 0 - moviment

Mentre la variable apropiada pren el valor *false*, el robot es mou de manera lineal en l'eix x. Quan canvia a *true*, s'atura i passa al següent estat.

Estat 1 – detecció

Amb el robot aturat, s'espera a la classificació de l'obstacle detectat (a `scanCallback`) i s'actua en funció del resultat.

Si s'arriba a la conclusió que es tracta d'una paret, es passa a estat 2.

Si, en canvi, s'observa un obstacle individual, es fa el càlcul de la posició global d'aquest. Concretament, fem ús de la trigonometria per passar de coordenades relatives a globals, tal com es descriu a continuació:

Càlcul de les posicions globals d'objectes

A partir de l'odometria obtenim la posició i orientació actuals del robot:

$x = \text{current.pose.x}$

$y = \text{current.pose.y}$

$\theta = \text{current.pose.theta}$

Mitjançant el sensor del làser obtenim la distància del robot a l'obstacle:

$\text{dist} = \text{valor de msg} \rightarrow \text{ranges}[i]$

L'objectiu és calcular les coordenades globals de l'obstacle detectat:

$\text{obs_x} = x + x'$

$\text{obs_y} = y + y'$

Per trobar el valor d' x' i y' utilitzem trigonometria:

$x' = \text{dist} \times \cos(\theta + \alpha)$

$y' = \text{dist} \times \sin(\theta + \alpha)$

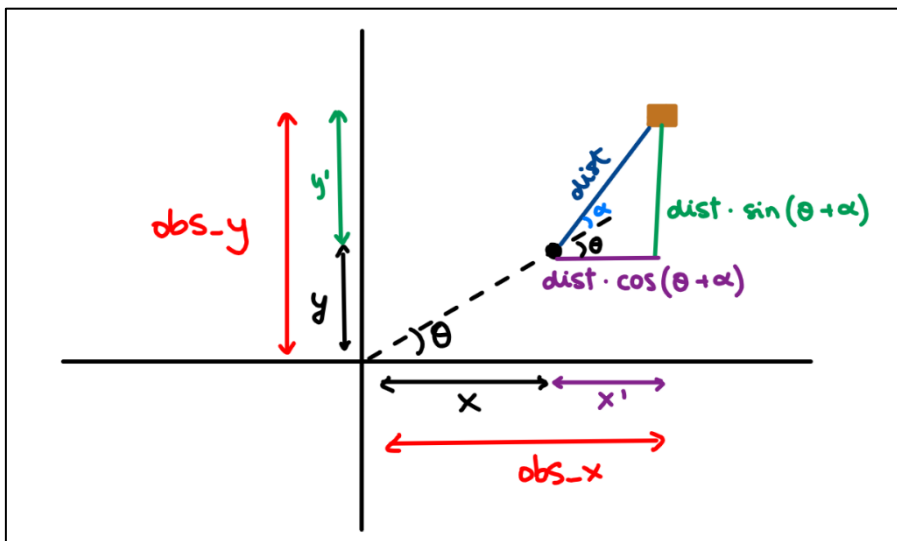
(però a segueix essent una incògnita)

Finalment:

$\text{obs_x} = \text{current.pose.x} + \text{dist} \times \cos(\text{current.pose.theta} + \alpha)$

$\text{obs_y} = \text{current.pose.y} + \text{dist} \times \sin(\text{current.pose.theta} + \alpha)$

El següent esquema resumeix els càlculs esmentats:



Càlcul de l'angle (α) – scanCallback

Aquest **angle** és la direcció en què s'ha detectat l'objecte respecte al davant del robot i es calcula a l'scanCallback de la següent manera:

$\text{angle} = (\text{msg->angle_min}) + i * (\text{msg->angle_increment})$

- angle_min és l'angle (en radians) on comença el primer raig del làser.
- angle_increment és la separació entre dos raigs consecutius del làser, en el nostre robot és aproximadament 1 grau ($2\pi / 360 = 0.01745$ rad).
- i és l'índex o grau on s'ha trobat l'obstacle.

El valor d'aquestes variables l'hem obtingut a partir de la informació del missatge **sensor_msgs/LaserScan**, que és el que utilitza el nostre turtlebot3.

Així doncs, un cop obtingudes les variables globals, es comprova si l'obstacle és conegut o nou, gràcies al vector **obstacles_coneguts**. Aquest desa la posició dels diferents obstacles quan són detectats per primera vegada.

En implementar aquesta funcionalitat, ha estat essencial buscar una manera de gestionar l'inevitable error de precisió en el càlcul de les coordenades.

Concretament, hem optat per considerar un rang d'error en la identificació d'obstacles. Qualsevol parell de coordenades a una distància inferior al llindar definit, es reconeix com un mateix obstacle. D'aquesta manera, no només resollem el problema de la manca de precisió, sinó que aconseguim que un obstacle detectat des de diferents orientacions pugui ser reconegut com un únic.

Seguint aquesta lògica, es recorre el vector mencionat i es comprova la distància entre les coordenades. Si alguna es troba a una distància inferior a l'error, es passa a l'estat 3. En canvi, si la posició és la d'un obstacle nou, es passa a l'estat 4.

Estat 2 i 3 – detecció de paret o obstacle conegut

Si el robot es troba en una d'aquestes dues situacions, el seu objectiu serà tornar tan aviat com es pugui a l'estat 0, moviment, per continuar explorant l'entorn.

Per fer-ho, hem implementat una lògica de **obstacle avoider** simple. En particular, es manté una velocitat angular fins que la variable aprop deixa de prendre el valor *true*. És a dir, el robot gira fins que pot continuar la trajectòria frontal sense obstacles.

Estat 4 – detecció d'un obstacle nou

L'última situació a considerar és quan el robot es troba davant d'un obstacle per primera vegada. És llavors quan ha de realitzar la seva tasca d'inspector.

En primer lloc, es desen les coordenades globals al vector **obstacles_coneguts**. Seguidament, el robot realitza una volta sencera sobre si mateix que representa simbòlicament el moment de la inspecció.

Per realitzar aquesta volta, hem decidit calcular l'angle relatiu girat en cada iteració per tal d'acumular-los fins a completar el gir sencer.

Concretament, per cada spin de ros es calcula l'angle actual mitjançant la odometria i resta l'angle anterior per trobar l'increment. Acumulem aquest gir relatiu a la variable `angle_girat` (que és el que fem servir per saber quan s'acaba la volta) en valor absolut, i l'angle anterior pren el valor de l'angle actual per la següent iteració. Per solucionar el problema de ROS de passar de π a $-\pi$, quan és necessari, sumem o restem 2π (360 graus).

A més a més, hem optat per considerar un gir una mica major, de 60 graus extra, per evitar que el robot finalitzi la inspecció just davant de l'obstacle. Això, permet tornar de manera immediata a l'estat d'exploració i evita que es torni a detectar l'obstacle immediatament inspeccionat, el que podria portar a bucles conflictius.

Un cop finalitzada la inspecció, es recupera l'estat 0.

Gràcies a aquesta lògica per estats aconseguim un codi clar i estructurat que garanteix el funcionament del robot segons la situació en què es troba, així com tractar la informació del sensor i l'odometria correctament.

Simulació en Gazebo

Durant la simulació del programa en Gazebo hem pogut observar que el robot fa servir un sistema de coordenades amb els eixos de referència girats 90 graus en sentit antihorari. Això no afecta de cap manera els càlculs ni funcionalitats, però pot ser confús a l'hora de visualitzar els missatges durant l'execució.

A continuació, proporcionem un esquema que pot ajudar a facilitar la comprensió de les coordenades. En concret, hem marcat les posicions aproximades dels obstacles del mapa *world*, que és el que hem optat per utilitzar durant les proves de simulació.

